# Full-Gradient Successor Feature Learning for Multi-Task Transfer in Reinforcement Learning

Ritish Shrirao
Dept. of CSE, IIIT Bangalore
ritish.shrirao@iiitb.ac.in

Aditya Priyadarshi
Dept. of CSE, IIIT Bangalore
aditya.priyadarshi@iiitb.ac.in

Raghuram Bharadwaj Diddigi
Dept. of DSAI, IIIT Bangalore
raghuram.bharadwaj@iiitb.ac.in

***Abstract*— Successor Features (SFs) provide a principled framework for multi-task transfer in Reinforcement Learning (RL) by decoupling environment dynamics from reward functions. By learning representations of expected future feature occupancies, SFs enable knowledge acquired in one task to be transferred to other tasks with different reward functions. However, conventional SF learning relies on semi-gradient Temporal Difference (TD) updates, which, when combined with nonlinear function approximation, generally lack convergence guarantees and can exhibit instability. This motivates exploring alternative learning objectives that offer stronger theoretical properties. Inspired by Full-Gradient DQN, we propose Full-Gradient Successor Feature Representations Q-Learning (FG-SFRQL), which extends full-gradient optimization to successor feature learning by minimizing the squared SF Bellman residual and differentiating through both the prediction and the bootstrapped target. We establish almost-sure convergence of FG-SFRQL under randomized task sampling, characterize the objective it minimizes, and show that averaging the bootstrapped target over $N$ next-state transitions reduces double-sampling bias in proportion to $1/N$. Through experiments in both discrete- and continuous-state domains, we demonstrate that full-gradient learning achieves lower Bellman residuals and improved transfer performance compared to semi-gradient baselines, with FG-SFRQL attaining the highest final return across all five evaluated domains.**[1]

## I. Introduction

Deep Reinforcement Learning (RL) [1], with landmark results such as DQN [2], enables agents to master high-dimensional control tasks. Most RL agents, however, are trained to optimize a single, fixed reward function, resulting in specialized policies that generalize poorly: when the task changes even slightly, they must often be retrained from scratch, wasting data and computation.

To address this limitation, researchers have explored representations that disentangle environment dynamics from task-specific rewards. A prominent approach is the Successor Representation (SR) [3], which encodes the expected discounted future state occupancy under a given policy. Building on this idea, Successor Features (SFs) [4] extend the SR to feature-based representations, decomposing the value function into a predictive model of future feature activations (capturing dynamics) and a task-dependent weight vector (capturing rewards). In doing so, SFs provide a natural bridge between model-free and model-based RL [5]: agents can predict future reward outcomes, as model-based planners do, while retaining the efficiency of model-free learning.

SFs have also been used for unsupervised pre-training [6], exploration [7], safety-constrained planning [8] and inverse RL [9]. Despite this widespread adoption, current deep SF methods largely rely on standard Q-learning mechanisms [2] to learn the successor features. From an optimization standpoint, these are *semi-gradient* methods: they treat the bootstrapped target as a constant during the update, ignoring its dependence on the trainable parameters. While computationally convenient, semi-gradient methods combined with non-linear function approximation can diverge or oscillate [10], [11]. This issue is especially important for transfer, where SFs are paired with Generalized Policy Improvement (GPI) [4] to reuse knowledge across tasks. GPI evaluates previously learned policies under new reward functions and assumes that the learned SFs accurately capture expected future feature occupancy. If the optimization procedure fails to converge, these estimates become unreliable, undermining GPI's guarantees and degrading transfer performance.

To address these limitations, we introduce Full-Gradient Successor Feature Representations Q-Learning (FG-SFRQL). Drawing inspiration from the unified view of TD algorithms of [12] and from Full-Gradient DQN [13], we formulate SF learning as the direct minimization of a squared Bellman residual, differentiating through both the prediction and the target. Because single-sample residual minimization suffers from double-sampling bias in stochastic environments [10], we also introduce an averaged variant that averages the bootstrapped target over $N$ independent next-state transitions [13].

Our contributions are as follows:

1) We propose FG-SFRQL, an algorithm for learning successor features that replaces the semi-gradient update of SFR Q-learning (SFRQL) [14] with the full gradient of the squared SF Bellman residual, together with a target-averaged variant that reduces the double-sampling bias of single-sample residual minimization.
2) Building on the analysis of Full-Gradient DQN [13], we cast the randomized-task variants of FG-SFRQL as stochastic recursive inclusions, prove almost-sure convergence of the iterates to an internally chain transitive set of the limiting differential inclusion, and show that the limit consists of stationary points of an explicit

[1]Supplementary material (Appendices A-D, containing proofs, experimental details, additional results) and code available at: `https://github.com/RitishShrirao/Full-Gradient-Successor-Feature-Representations`.

aggregate SF Bellman objective whenever it is free of ties in the GPI and target-action selections.

3) We show empirically that FG-SFRQL achieves lower Bellman residuals and outperforms semi-gradient baselines across all five domains with negligible computational overhead, and that averaging the target over $N$ next-state samples further boosts performance under stochastic transitions, as predicted by the analysis.

## II. Related Work

### A. Successor Features and Transfer Learning

The Successor Representation (SR), introduced by Dayan [3], decomposes the value function into a predictive occupancy map and a local reward vector. Barreto et al. [4] generalized this to Successor Features (SFs) and introduced Generalized Policy Improvement (GPI), establishing a framework for zero-shot transfer across tasks with shared dynamics. Reinke and Alameda-Pineda [14] extended them to non-linear rewards (SFR), the formulation we adopt.

### B. Gradient-Based Optimization and Stability in RL

Semi-gradient Temporal Difference (TD) learning can diverge when combined with non-linear function approximation and off-policy sampling, the "deadly triad" [1], [11], [15]. Baird's residual algorithms [10] perform true stochastic gradient descent on the Bellman residual; while stable, they were criticized for their double-sampling requirement and slow convergence.

Loth and Preux [12] proposed a unified view of TD algorithms, showing that methods ranging from TD($\lambda$) to residual gradients can be understood as minimizing specific objective functions. More recently, Avrachenkov et al. [13] identified theoretical shortcomings of the standard DQN update through an o.d.e. analysis and proposed Full-Gradient DQN (FG-DQN), a modified and provably convergent stochastic approximation scheme. Our work extends the FG-DQN formalism to the multi-task successor-feature setting, deriving update rules that account for the vector-valued nature of SFs and for the max-over-policies operator inherent in GPI.

### C. Convergence Analysis of Successor Features

The convergence of SFs under function approximation has received little analysis. Zhang et al. [16] analyze SF-DQN, but in the semi-gradient regime and under strong assumptions on initialization and exploration.

In contrast, we frame SF learning as a stochastic recursive inclusion driven by the full gradient of the SF Bellman residual. Because the update is a (generalized) gradient of a fixed objective, the analysis does not require controlling a moving target, and we obtain almost-sure convergence under standard stability and regularity conditions, which we state explicitly in Section VI.

## III. Background

### A. Successor Feature Representations (SFR)

Successor Feature Representations (SFR) [14] handle arbitrary, potentially non-linear rewards. For a policy $\pi$, the $\xi$-function is the cumulative discounted probability of observing feature $\phi \in \Phi$ in the future:

$$\xi^{\pi}(s,a,\phi) = \sum_{k=0}^{\infty} \gamma^k P(\phi_{t+k} = \phi \mid s_t = s, a_t = a, \pi), \quad (1)$$

where $\phi_t = \phi(s_t, a_t, s_{t+1})$ and $\gamma \in [0,1)$ is the discount factor. If the reward $R(\phi)$ depends only on the features, the action-value function is recovered by aggregating the $\xi$-values:

$$Q^{\pi}(s,a) = \sum_{\phi \in \Phi} \xi^{\pi}(s,a,\phi) R(\phi). \quad (2)$$

This allows an agent to evaluate an existing policy $\pi$ under a *new* reward function $R'$ by replacing $R$ with $R'$, without new environment interaction.

### B. Generalized Policy Improvement (GPI)

The primary transfer mechanism in this framework is GPI. Suppose an agent has learned a library of policies $\Pi = \{\pi_1, \ldots, \pi_n\}$ for previous tasks. When facing a new task with reward $R_{\text{new}}$, the agent evaluates *all* stored policies using their $\xi$-functions:

$$Q^{\pi_i}_{\text{new}}(s,a) = \sum_{\phi \in \Phi} \xi^{\pi_i}(s,a,\phi) R_{\text{new}}(\phi). \quad (3)$$

GPI then acts greedily with respect to the best of these values:

$$\pi_{\text{GPI}}(s) \in \arg\max_a \max_i Q^{\pi_i}_{\text{new}}(s,a). \quad (4)$$

The GPI policy performs at least as well as every policy in the library [4]. Extensions such as constrained GPI [17] further refine this selection by incorporating constraints or approximate models.

## IV. Algorithmic Setup and Notation

We follow notations similar to [14]. We assume a finite state space $\mathcal{S}$, a finite action set $\mathcal{A}$ and a finite feature set $\Phi$. There are $m$ tasks indexed by $j \in \{1, \ldots, m\}$. All tasks share the transition kernel $P(\cdot \mid s,a)$ and the feature map $\phi(s,a,s') \in \Phi$; they differ only in their reward $\mathcal{R}_j(\phi)$. For each task $j$ the agent maintains a parametric successor-feature approximator $\xi_j(s,a,\phi;\theta^{(j)})$ with $\theta^{(j)} \in \mathbb{R}^d$, and we collect all task parameters into the joint vector $\Theta = (\theta^{(1)}, \ldots, \theta^{(m)}) \in \mathbb{R}^{md}$. We write $\gamma(s') = 0$ if $s'$ is terminal and $\gamma(s') = \gamma$ otherwise.

The value of policy $j$ under the reward of task $i$ is

$$Q^i_j(s,a;\Theta) = \sum_{\phi \in \Phi} \xi_j(s,a,\phi;\theta^{(j)})\, \mathcal{R}_i(\phi). \quad (5)$$

At each update the current task $i$ provides a transition $(s,a,s')$. The algorithm selects a prior policy $c$ by GPI over a library $\mathcal{I}_i$ of policies, choosing from the set

$$\mathcal{C}(\Theta,s,i) = \arg\max_{j \in \mathcal{I}_i} \max_{a \in \mathcal{A}} Q^i_j(s,a;\Theta). \quad (6)$$

**Algorithm 1** Joint Full-Gradient Model-Free SFRQL (FG-SFRQL); full pseudocode in Appendix D

**Input:** exploration rate $\epsilon$; step sizes $\{\alpha_t\}$; reward learning rate $\alpha_R$; features $\Phi$

**for** $i = 1$ to $m$ **do**
  Initialize $\theta_i^R$ randomly (unless $\mathcal{R}_i$ is given)
  $\theta^{(i)} \leftarrow \theta^{(i-1)}$ ($\theta^{(1)}$ random)
  **for** $t = 1$ to num_steps **do**
    At the start of an episode, $s_t \leftarrow$ initial state
    $c \leftarrow$ GPI prior (6) at $s_t$, $\mathcal{I}_i = \{1, \dots, i\}$
    $a_t \leftarrow \epsilon$-greedy action of $Q_c^i(s_t, \cdot)$
    Take $a_t$; observe $r_t$, $s_{t+1}$ and $\phi_t = \phi(s_t, a_t, s_{t+1})$
    $\theta_i^R \leftarrow \mathrm{SGD}(\alpha_R)$ step on $(r_t - \tilde{\mathcal{R}}_i(\phi_t))^2$
    $\gamma_t \leftarrow \gamma(s_{t+1})$; end the episode if $s_{t+1}$ is terminal
    $b_i \leftarrow \arg\max_{a'} \max_{k \le i} Q_k^i(s_{t+1}, a')$ ▷ GPI
    $\theta^{(i)} \leftarrow \theta^{(i)} - \alpha_t \nabla_{\theta^{(i)}} \ell_i$ ▷ full gradient
    **if** $c \neq i$ **then**
      $b_c \leftarrow \arg\max_{a'} Q_c^c(s_{t+1}, a')$ ▷ own greedy
      $\theta^{(c)} \leftarrow \theta^{(c)} - \alpha_t \nabla_{\theta^{(c)}} \ell_c$
    $s_t \leftarrow s_{t+1}$

$Q_k^i$ is (5) with the learned reward $\tilde{\mathcal{R}}_i$; $\ell_j$ is (8) with $N = 1$, pivot $(s_t, a_t)$, next state $s_{t+1}$ and target action $b_j$, and its full gradient is (10), with the factor 2 absorbed into $\alpha_t$.

In the sequential Algorithm 1 the library holds the tasks seen so far, $\mathcal{I}_i = \{1, \dots, i\}$; in the randomized Algorithms 2 and 3 it holds all $m$ tasks.

The algorithm updates the parameter block of the current task $i$ and, if $c \neq i$, the block of the prior $c$; all other blocks are left unchanged. As in SFQL [4], the two blocks bootstrap with different next actions: the current task uses the GPI action $\arg\max_{a'} \max_{j \in \mathcal{I}_i} Q_j^i(s', a')$, while the prior $c$ uses its own greedy action $\arg\max_{a'} Q_c^c(s', a')$ under its own reward. We denote the set of these target actions for block $j$ by $\mathcal{B}_j^i(\Theta, s')$.

If a maximizer is not unique, ties are broken by a fixed rule (e.g., smallest index). The analysis also covers arbitrary tie-breaking by considering the convex hull of the update vectors of all maximizers (Section VII). We call $\Theta$ *regular* if no ties occur in any of the GPI and target-action maximizations, and denote the set of regular parameters by $\mathcal{U}$ and the tie set by $\mathcal{T} = \mathbb{R}^{md} \setminus \mathcal{U}$. In a neighborhood of a regular point, all selections are locally constant.

Algorithm 1 is shown in compact form below; the full pseudocode of the three variants of FG-SFRQL is given in Appendix D:

- **Algorithm 1** is the sequential implementation used in the experiments, where tasks are learned one after another. It is a direct modification of SFRQL that replaces the semi-gradient update with the full gradient.
- **Algorithm 2** samples the task from a distribution $\rho$ and the state-action pair (the *pivot*) from a task-dependent distribution $\mu_i$, so that the sampling is stationary, as required by the mean-field analysis.
- **Algorithm 3** extends Algorithm 2 by *averaging* over $N$ next-state samples from the same pivot.

## V. Averaged Loss and Full Gradient

With stochastic transitions, minimizing the single-sample Bellman residual yields a biased objective. Following FG-DQN [13], we therefore average the target over several next states. For a pivot $(s, a)$ and current task $i$, let $\mathcal{K} = \{s'_p\}_{p=1}^N$ be $N$ next states sampled from $P(\cdot \mid s, a)$, with features $\phi_p = \phi(s, a, s'_p)$ and discounts $\gamma_p = \gamma(s'_p)$. For each sample, a target action $b_{j,p} \in \mathcal{B}_j^i(\Theta, s'_p)$ is chosen as described above. We define the *averaged* Bellman error of block $j$ at feature $\phi$ as

$$\bar{\delta}_j(\phi) = \frac{1}{N} \sum_{p=1}^{N} \Big[ \mathbb{I}(\phi = \phi_p) + \gamma_p \, \xi_j(s'_p, b_{j,p}, \phi; \theta^{(j)}) \Big] - \xi_j(s, a, \phi; \theta^{(j)}), \quad (7)$$

and the averaged squared Bellman residual as

$$\ell_j = \sum_{\phi \in \Phi} \bar{\delta}_j(\phi)^2. \quad (8)$$

This loss measures the distance between the current prediction and the *empirical mean* of the target, which approaches the true expectation $\mathbb{E}[\cdot \mid s, a]$ as $N \to \infty$. For $N = 1$ it is the single-sample squared residual used in Algorithms 1 and 2; $N > 1$ gives Algorithm 3, and Algorithm 1 with $N > 1$ is the *averaged variant* of Algorithm 1 used in Section IX-C.3 and Appendix C.5.

Following the Full-Gradient DQN scheme, we differentiate with respect to $\theta^{(j)}$ in *both* the target and the prediction, holding the target actions fixed. With the averaged feature gradient

$$\bar{D}_j(\phi) = \frac{1}{N} \sum_{p=1}^{N} \gamma_p \nabla_{\theta^{(j)}} \xi_j(s'_p, b_{j,p}, \phi; \theta^{(j)}) - \nabla_{\theta^{(j)}} \xi_j(s, a, \phi; \theta^{(j)}), \quad (9)$$

the full gradient of the loss is

$$\nabla_{\theta^{(j)}} \ell_j = 2 \sum_{\phi \in \Phi} \underbrace{\bar{\delta}_j(\phi)}_{\text{Bellman error}} \underbrace{\bar{D}_j(\phi)}_{\text{feature gradient}}. \quad (10)$$

A semi-gradient method would keep only the second term of $\bar{D}_j(\phi)$. The joint update vector $G$ uses this gradient in the appropriate blocks. If the current task is $i$ and the chosen prior is $c$, the $j$-th block of $G$ is

$$G_j = \begin{cases} \nabla_{\theta^{(i)}} \ell_i & \text{if } j = i, \\ \nabla_{\theta^{(c)}} \ell_c & \text{if } j = c \text{ and } c \neq i, \\ \mathbf{0} & \text{otherwise.} \end{cases} \quad (11)$$

The target action is chosen separately for each sample $s'_p$: selecting a single action for the averaged vector $\frac{1}{N} \sum_p \xi(s'_p, \cdot)$ would replace $\mathbb{E}[\max(\cdot)]$ by $\max \mathbb{E}[\cdot]$, and the resulting target would no longer correspond to the SF Bellman operator.

*What the update minimizes:* Averaging the loss over tasks $i \sim \rho$, pivots $(s, a) \sim \mu_i$ and batches $\mathcal{K}$ gives the objective

$$E^{(N)}(\Theta) = \mathbb{E}\Big[ \ell_i + \mathbb{I}\{c \neq i\} \, \ell_c \Big]. \quad (12)$$

The following proposition relates this objective to the true Bellman error.

**Proposition 1** (Bias–variance decomposition)**.** *For every* $\Theta$ *and* $N \geq 1$*,*

$$E^{(N)}(\Theta) = E^{\mathrm{BE}}(\Theta) + \tfrac{1}{N} V(\Theta), \tag{13}$$

*where* $E^{\mathrm{BE}}$ *is the aggregate mean-squared SF Bellman error (the objective* (12) *with each per-sample target replaced by its expectation over* $s'$*), and* $V \geq 0$ *is the corresponding conditional variance of the target.*

The proof is given in Appendix A (Lemma 3). For a single sample ($N = 1$), the objective exceeds the Bellman error by the conditional variance of the target, which is the well-known double-sampling bias of residual methods [10]. Averaging over $N$ samples (Algorithm 3) reduces this bias to $V/N$, and $E^{\mathrm{BE}}$ is recovered in the limit $N \to \infty$. Under deterministic dynamics, $V = 0$ and all $N$ samples are identical, so that averaging has no effect.

*Comparison with FG-DQN:* In FG-DQN, the bootstrapped target $\max_b Q(s', b; \theta)$ is a continuous function of $\theta$, since it is the maximum of finitely many continuous functions. In the SF setting, by contrast, the target is the *vector* $\xi_j(s', b, \cdot)$ evaluated at a greedy action $b$, and this vector changes discontinuously whenever the greedy action switches. Consequently, the objective $E^{(N)}$ is smooth only away from the tie set $\mathcal{T}$. This motivates the set-valued analysis of Section VII and the restriction of Theorem 1(ii) to tie-free limit sets.

## VI. Assumptions

**Assumption 1** (Properties of the system)**.** *The sets* $\mathcal{S}$*,* $\mathcal{A}$*,* $\Phi$ *and the number of tasks* $m$ *are finite,* $\gamma \in [0, 1)$*, and the task rewards* $\mathcal{R}_i$ *are known. For every* $(s, a, \phi, j)$*, the map* $\theta \mapsto \xi_j(s, a, \phi; \theta)$ *is twice continuously differentiable, so* $\xi_j$ *and its first two derivatives are bounded on compact sets.*

**Assumption 2** (Sampling)**.** *At each iteration* $k$*, given the past, the task is drawn as* $i_k \sim \rho$*, the pivot as* $(s_k, a_k) \sim \mu_{i_k}$*, and the batch* $\mathcal{K}_k$ *of* $N_k$ *next states i.i.d. from* $P(\cdot \mid s_k, a_k)$*. The distributions* $\rho$ *and* $\mu_i$ *do not depend on* $\Theta$*. The batch size is either fixed,* $N_k \equiv N$*, or grows,* $N_k \to \infty$ *(Algorithm 2 is the case* $N_k \equiv 1$*). We write* $\mathcal{F}_k$ *for the history up to iteration* $k$*.*

**Assumption 3** (Step sizes)**.** *The step sizes satisfy* $\alpha_k > 0$*,* $\sum_k \alpha_k = \infty$ *and* $\sum_k \alpha_k^2 < \infty$*.*

**Assumption 4** (Stability of iterates)**.** *The iterates remain bounded almost surely:* $\sup_k \|\Theta_k\| < \infty$ *a.s.*

Let $E := E^{(N)}$ if $N_k \equiv N$ and $E := E^{\mathrm{BE}}$ if $N_k \to \infty$, and let $\Lambda$ be the set of regular stationary points of $E$, i.e., $\Theta \in \mathcal{U}$ with $\nabla E(\Theta) = 0$.

**Assumption 5** (Stationary points)**.** *The set of values* $\{E(\Theta) : \Theta \in \Lambda\}$ *contains no interval; for example,* $\Lambda$ *is finite or countable.*

Assumptions 1–3 are standard in stochastic approximation and RL literature [13], [14], [18], [19]. Assumption 4 is also standard [19] and can be enforced by projecting the iterates onto a large ball. Assumption 5 is weaker than requiring finitely many stationary points, which is relevant for neural networks, whose symmetries typically give rise to non-isolated stationary points. In contrast to FG-DQN, it does not suffice here to assume that ties occur only on a set of parameters of measure zero, because the limiting dynamics can slide along such a set. We therefore state the final conclusion for limit sets that contain no ties.

## VII. Set-Valued Mean Field

Let $\omega = (i, s, a, \mathcal{K})$ denote one sample. When there are no ties, the update (11) is a single vector. When there are ties, each admissible choice $\sigma$ of the prior $c$ and of the target actions $b_{j,p}$ gives an update vector $G^\sigma(\Theta, \omega)$, and we collect them in their convex hull. The single-sample and averaged mean-field maps are

$$\begin{aligned} F^{(N)}(\Theta, \omega) &= \mathrm{co}\big\{G^\sigma(\Theta, \omega) : \sigma \in \Sigma(\Theta, \omega)\big\}, \\ \overline{H}^{(N)}(\Theta) &= \mathbb{E}_\omega\big[F^{(N)}(\Theta, \omega)\big], \end{aligned} \tag{14}$$

where $\Sigma(\Theta, \omega)$ is the finite set of admissible choices. The population map $\overline{H}^{(\infty)}$ is defined in the same way, with the sample averages in (8)–(10) replaced by expectations under $P(\cdot \mid s, a)$.

By Assumption 1, each $G^\sigma$ is continuous in $\Theta$, and a small perturbation of $\Theta$ can only remove ties, not create new ones. Therefore $\overline{H}^{(N)}$ is nonempty, compact, convex-valued, upper semicontinuous and bounded on compact sets. On the regular set $\mathcal{U}$ the maps reduce to single gradients: $\overline{H}^{(N)} = \{\nabla E^{(N)}\}$ and $\overline{H}^{(\infty)} = \{\nabla E^{\mathrm{BE}}\}$ (Lemma 3, Appendix A).

## VIII. Main Result

We adapt the o.d.e. analysis of Full-Gradient DQN [13] to successor features, where (i) the Bellman target is vector-valued and evaluated at a greedy action, (ii) GPI maximizes over both actions and policies, and (iii) the updates are coupled across tasks, using set-valued stochastic approximation [18], [19].

**Theorem 1** (Convergence of FG-SFRQL)**.** *Let* $\{\Theta_k\}$ *be generated by Algorithm 2 or 3, and let Assumptions 1–4 hold. Let* $\overline{H} = \overline{H}^{(N)}$ *if* $N_k \equiv N$ *and* $\overline{H} = \overline{H}^{(\infty)}$ *if* $N_k \to \infty$*. Then, almost surely:*

(i) *the iterates converge to a compact, connected, internally chain transitive (ICT) set* $L$ *of the differential inclusion*

$$\dot{\Theta}(t) \in -\overline{H}(\Theta(t)); \tag{15}$$

(ii) *if, in addition, Assumption 5 holds and* $L$ *contains no ties, i.e.,* $L \subseteq \mathcal{U}$*, then* $L$ *consists of stationary points of* $E$*,* $E(\Theta_k)$ *converges and* $\Theta_k$ *approaches* $\Lambda$*. If* $\Lambda$ *is finite,* $\Theta_k \to \Theta^*$ *with* $\nabla E(\Theta^*) = 0$*.*

Informally, an ICT set is invariant under the limiting dynamics, and any two of its points can be connected by trajectories that remain in the set. Theorem 1 thus states that FG-SFRQL asymptotically behaves like gradient descent on $E$ wherever the GPI selections are unambiguous.

The proof relies on the following results on stochastic approximation with set-valued maps.

**Lemma 1** (Benaïm, Hofbauer and Sorin [18]). *Let $F$ be a set-valued map on $\mathbb{R}^{md}$ whose graph is closed, whose values $F(x)$ are nonempty, compact and convex, and which satisfies $\sup_{z\in F(x)}\|z\| \le K(1+\|x\|)$ for some $K>0$. Let $\{x_k\}$ satisfy*

$$x_{k+1} - x_k - \alpha_k U_{k+1} \in \alpha_k F(x_k),$$

*where $\alpha_k > 0$, $\sum_k \alpha_k = \infty$, $\alpha_k \to 0$, and $U_{k+1}$ are (random) perturbations. Let $t_k = \sum_{n<k}\alpha_n$, and let $w : [0,\infty) \to \mathbb{R}^d$ be the piecewise-linear interpolation of the iterates on this time scale,*

$$w(t) = x_k + \frac{t-t_k}{\alpha_k}(x_{k+1}-x_k), \qquad t\in[t_k,t_{k+1}],$$

*so that $w(t_k) = x_k$ and $w$ is continuous. Its limit set is $L(w) = \bigcap_{t\ge 0}\overline{\{w(s) : s\ge t\}}$, the set of all limits of $w(s_n)$ along sequences $s_n\to\infty$.*

(a) *If $\sup_k\|x_k\|<\infty$ and, for every $T>0$,*

$$\lim_{n\to\infty}\sup\Big\{\Big\|\sum_{i=n}^{k-1}\alpha_i U_{i+1}\Big\| : k>n,\ t_k\le t_n+T\Big\} = 0,$$

*then $L(w)$ is internally chain transitive for $\dot x \in F(x)$.*

(b) *Every internally chain transitive set $L$ is invariant: through every point of $L$ passes a solution of $\dot x\in F(x)$ that stays in $L$ for all times.*

(c) *Let $L$ be an internally chain transitive set, let $\Lambda\subseteq\mathbb{R}^{md}$, and let $V$ be a function that is continuous on $L$. Suppose that for every solution $x$ and every $t>0$ with $x([0,t])\subseteq L$, $V(x(t)) < V(x(0))$ if $x(0)\notin\Lambda$ and $V(x(t))\le V(x(0))$ if $x(0)\in\Lambda$. If $V(\Lambda)$ has empty interior, then $L\subseteq\Lambda$ and $V$ is constant on $L$ (proof in Appendix A).*

*Proof.* The proof follows the o.d.e. approach for stochastic approximation, adapted to set-valued maps and differential inclusions [18], [19].

***Update rule:*** The update is $\Theta_{k+1} = \Theta_k - \alpha_k\tilde G_k$, where $\tilde G_k$ has at most two non-zero blocks, for the current task $i_k$ and the GPI-selected prior $c_k$. We split $\tilde G_k$ into its conditional mean and a noise term. By Lemma 3, the conditional mean $h(\Theta_k) = \mathbb{E}[\tilde G_k \mid \mathcal{F}_k]$ is a selection from $\overline{H}^{(N_k)}(\Theta_k)$. Let $M_{k+1} = \tilde G_k - h(\Theta_k)$ be the noise and, when $N_k\to\infty$, let $\varepsilon_k$ be the difference between the batch and population mean fields ($\varepsilon_k = \mathbf{0}$ for fixed $N$). Then

$$\Theta_{k+1} \in \Theta_k - \alpha_k\big(\overline{H}(\Theta_k) + \varepsilon_k + M_{k+1}\big), \qquad (16)$$

which is a stochastic recursive inclusion.

***Noise and bias:*** By Lemma 2 (Appendix A), $M_{k+1}$ is a martingale difference sequence, and for every $R>0$ there is a constant $C_R<\infty$, depending only on $R$ and the bounds of Assumption 1, such that on $\{\|\Theta_k\|\le R\}$ its conditional second moment is at most $C_R$. Since $\mathbb{I}\{\|\Theta_k\|\le R\}$ is $\mathcal{F}_k$-measurable, $\sum_k \alpha_k M_{k+1}\mathbb{I}\{\|\Theta_k\|\le R\}$ is a martingale whose increments have second moments summing to at most $C_R\sum_k\alpha_k^2<\infty$; it is bounded in mean square and converges almost surely. On the event $\{\sup_k\|\Theta_k\|\le R\}$ it equals $\sum_k\alpha_k M_{k+1}$. By Assumption 4, the union of these events over $R\in\mathbb{N}$ has probability one, and a countable union of null sets is null. Hence the noise series $\sum_k\alpha_k M_{k+1}$ converges almost surely. By Lemma 3(c), $\|\varepsilon_k\| = O(1/N_k)\to 0$. Since the iterates are bounded (Assumption 4), it suffices to argue on the event that they stay in a fixed ball, where $\overline{H}$ is bounded.

***Limiting differential inclusion:*** Write (16) as $\Theta_{k+1} - \Theta_k - \alpha_k U_{k+1} \in \alpha_k F(\Theta_k)$ with $F := -\overline{H}$ and $U_{k+1} := -(M_{k+1}+\varepsilon_k)$, which is the recursion of Lemma 1. On the event that the iterates stay in the closed ball $B_R = \{\Theta : \|\Theta\|\le R\}$, replace $\overline{H}$ by $\overline{H}\circ\pi_R$, where $\pi_R$ is the projection onto $B_R$; this changes neither the iterates nor the dynamics on $B_R$. The resulting map has a closed graph, nonempty compact convex values and bounded range, so it satisfies the conditions on $F$ in Lemma 1.

The perturbations satisfy the condition of Lemma 1(a), i.e., $\sum_{i=n}^{k-1}\alpha_i U_{i+1}\to 0$ uniformly over windows $t_k - t_n \le T$. For the noise this holds because $\sum_k\alpha_k M_{k+1}$ converges, so its tail sums vanish. For the bias it holds because the window sum is at most $(T+\alpha_n)\sup_{i\ge n}\|\varepsilon_i\|\to 0$. Since the iterates are bounded, Lemma 1(a) shows that the limit set $L := L(w)$ of the interpolation $w$ of $\{\Theta_k\}$ (with $x_k = \Theta_k$) is internally chain transitive for $\dot\Theta\in F(\Theta)$. Two further steps are needed to obtain statement (i).

First, $L$ is also the set of limit points of the iterates $\{\Theta_k\}$. The increments vanish: $\Theta_{k+1}-\Theta_k = -\alpha_k\big(h(\Theta_k) + \varepsilon_k + M_{k+1}\big)$, where $h(\Theta_k)$ is bounded on $B_R$, $\varepsilon_k\to 0$, $\alpha_k\to 0$, and $\alpha_k M_{k+1}\to 0$ because it is the general term of a convergent series. The interpolation passes through every iterate, $w(t_k) = \Theta_k$, so every limit point of $\{\Theta_k\}$ lies in $L$. Conversely, for $t\in[t_k,t_{k+1}]$ the point $w(t)$ lies on the segment from $\Theta_k$ to $\Theta_{k+1}$, so $\|w(t)-\Theta_k\|\le\|\Theta_{k+1}-\Theta_k\|\to 0$. Hence, if $w(s_n)\to x$ along some $s_n\to\infty$, the iterates at the corresponding indices also converge to $x$, and every point of $L$ is a limit point of $\{\Theta_k\}$. As the limit set of a bounded continuous path, $L$ is compact and connected.

Second, $L$ is ICT for the original inclusion (15), not only for the modified one. All iterates lie in the closed ball $B_R$, so $L\subseteq B_R$. On $B_R$ the projection is the identity, $\pi_R(\Theta)=\Theta$, so the modified map $-\overline{H}\circ\pi_R$ equals $-\overline{H}$ there. Internal chain transitivity of $L$ involves only solutions that stay in $L$: by definition, any two points of $L$ are joined by chains of solution segments that lie in $L$, with arbitrarily small jumps between segments. On $L$ the two maps are equal, so these segments are also solutions of (15), and $L$ is an ICT set of (15). Here ICT for (15) is meant in this sense of chains of solution segments in $L$, which does not require solutions of (15) to exist for all times; the remaining steps apply Lemma 1 to the modified map, which satisfies its conditions. This proves (i).

***Convergence analysis:*** Suppose now that $L$ contains no ties, i.e., $L\subseteq\mathcal{U}$. Since $\mathcal{U}$ is open and $L$ is compact, $L$ has a tie-free neighborhood, on which the selections are fixed, $E$ is twice continuously differentiable, and $\overline{H}(\Theta) = \{\nabla E(\Theta)\}$.

There, the differential inclusion reduces to the o.d.e.

$$\dot{\Theta} = -\nabla E(\Theta). \tag{17}$$

Along its trajectories, the aggregate error decreases:

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}E(\Theta(t)) &= \langle \nabla E(\Theta), \dot{\Theta}\rangle \\ &= \sum_{j=1}^{m} \left\langle \nabla_{\theta^{(j)}} E(\Theta), -\nabla_{\theta^{(j)}} E(\Theta)\right\rangle \\ &= -\sum_{j=1}^{m} \|\nabla_{\theta^{(j)}} E(\Theta)\|^2 \;\leq\; 0, \end{aligned} \tag{18}$$

with equality only at stationary points. Hence $E$ is a Lyapunov function for the limiting dynamics. On $L$ the modified map equals $-\nabla E$, so every solution segment that lies in $L$ solves $\dot{\Theta} = -\nabla E(\Theta)$. By (18), $E$ decreases strictly along such a segment if it starts outside $\Lambda$, and does not increase if it starts in $\Lambda$. Since $E$ is also continuous on the tie-free neighborhood of $L$, it satisfies the conditions of Lemma 1(c). Since the values of $E$ on $\Lambda$ contain no interval (Assumption 5), the lemma gives $L \subseteq \Lambda$ with $E$ constant on $L$. As the iterates approach $L$ and $E$ is continuous near $L$, $E(\Theta_k)$ converges and $\Theta_k$ approaches $\Lambda$. Finally, if $\Lambda$ is finite, the connected set $L$ must be a single point $\Theta^*$, and $\Theta_k \to \Theta^*$. □

The $j$-th block of $\nabla E$ collects the iterations in which $j$ is the current task *and* those in which it is the GPI-selected prior (Remark 1, Appendix A).

*Replay buffers and batch schedules:* With a replay buffer, the sampling laws are the empirical buffer laws rather than $\mu$ and $P$; the resulting error can be absorbed into $\varepsilon_k$ whenever the buffer distribution converges, e.g., under a fixed ergodic behavior policy. For growing batches, only $\varepsilon_k \to 0$ is needed, with no condition linking $N_k$ to $\alpha_k$.

## IX. Experiments

### A. Experimental Setup

We evaluate FG-SFRQL in five domains. In each domain, all tasks share the transition dynamics and differ only in their reward functions; Appendix B.1 gives the details.

- **Four Rooms** [20] (discrete features): a grid world in which the agent collects objects on its way to a goal; the 6 tasks assign different values to the objects.
- **Reacher** [21] (continuous features): a two-joint arm with 9 discretized torques; each of the 4 tasks has a different target location.
- **PointMaze** [22] (continuous features): a point agent with 9 discretized actions in the `UMaze`, `Medium` and `Large` layouts; each of the 8 tasks has a different goal location and a dense reward.

For the continuous-feature domains, whose rewards are linear in $\phi$, we use linear SFs $\psi_j(s,a) \in \mathbb{R}^{\dim \phi}$ (Appendix B.1), demonstrating that FG-SFRQL generalizes effectively to continuous state spaces beyond the finite-space setting of Assumption 1.

### B. Baselines, Protocol and Hyperparameters

We denote by SFDQN and FG-SFDQN the deep-network implementations of SFRQL and FG-SFRQL, respectively; unless stated otherwise, FG-SFDQN refers to Algorithm 1. We compare:

1) **DQN**, a single network trained on the tasks in sequence, and **FG-DQN** [13], its full-gradient version;
2) **SFDQN**, successor features with semi-gradient updates and GPI; like Algorithm 1, it updates the current task on every sample and each GPI-selected prior on the samples where it is selected [4];
3) **FG-SFDQN (ours)**: the sequential Algorithm 1, and the randomized variants Algorithm 2 (single sample) and Algorithm 3 (averaging over $N = 5$ next states).

The sequential methods train on each task for a fixed number of steps before proceeding to the next one. Algorithms 2 and 3 use the same total budget (steps per task $\times$ number of tasks), but draw the task uniformly at random at every step. Since the replay buffers of the DQN baselines store rewards, they are emptied at every task switch, whereas the SF methods store task-independent features and keep their buffers.

Hyperparameters are listed in Table I (Appendix B.2). Every configuration is run with 3 seeds, and we report mean $\pm$ standard deviation across seeds. After training on all tasks, each trained agent is evaluated on every task for 10 episodes of at most 100 steps; SF methods act by GPI over their learned successor features. Following standard deep RL practice, all methods use a constant learning rate and minibatch updates.

### C. Results and Analysis

*1) Training performance:* As shown in the upper panels of Fig. 1, FG-SFDQN (Algorithm 1) achieves the highest cumulative training reward across all five domains: it collects $2.7\times$ more reward than its semi-gradient counterpart SFDQN in Four Rooms (1365 vs. 500) and 9–13% more across all three PointMaze layouts, while matching SFDQN's high training return in Reacher (85.3k vs. 84.1k; full totals in Appendix C.1). As expected, sequential training (Algorithm 1) collects more reward during training than the randomized variants (Algorithms 2 and 3), since sequential training warm-starts $\theta^{(i)} \leftarrow \theta^{(i-1)}$ on a focused task whereas uniform task sampling distributes early exploration across all $m$ tasks simultaneously.

*2) Final evaluation:* The lower panels of Fig. 1 show the final evaluation return on every task after training completes (mean values are given in Appendix C.1). FG-SFDQN (Algorithm 1) achieves the highest mean return in all five domains, substantially outperforming all baselines: it improves over SFDQN by $8.2\times$ in Four Rooms (3.38 vs. 0.41), by 85% in Reacher (71.71 vs. 38.68), and by 24–39% across the three PointMaze layouts. Crucially, the randomized variants (Algorithms 2 and 3), which are directly covered by Theorem 1, rank second and third in every domain, outperforming SFDQN, DQN, and FG-DQN across

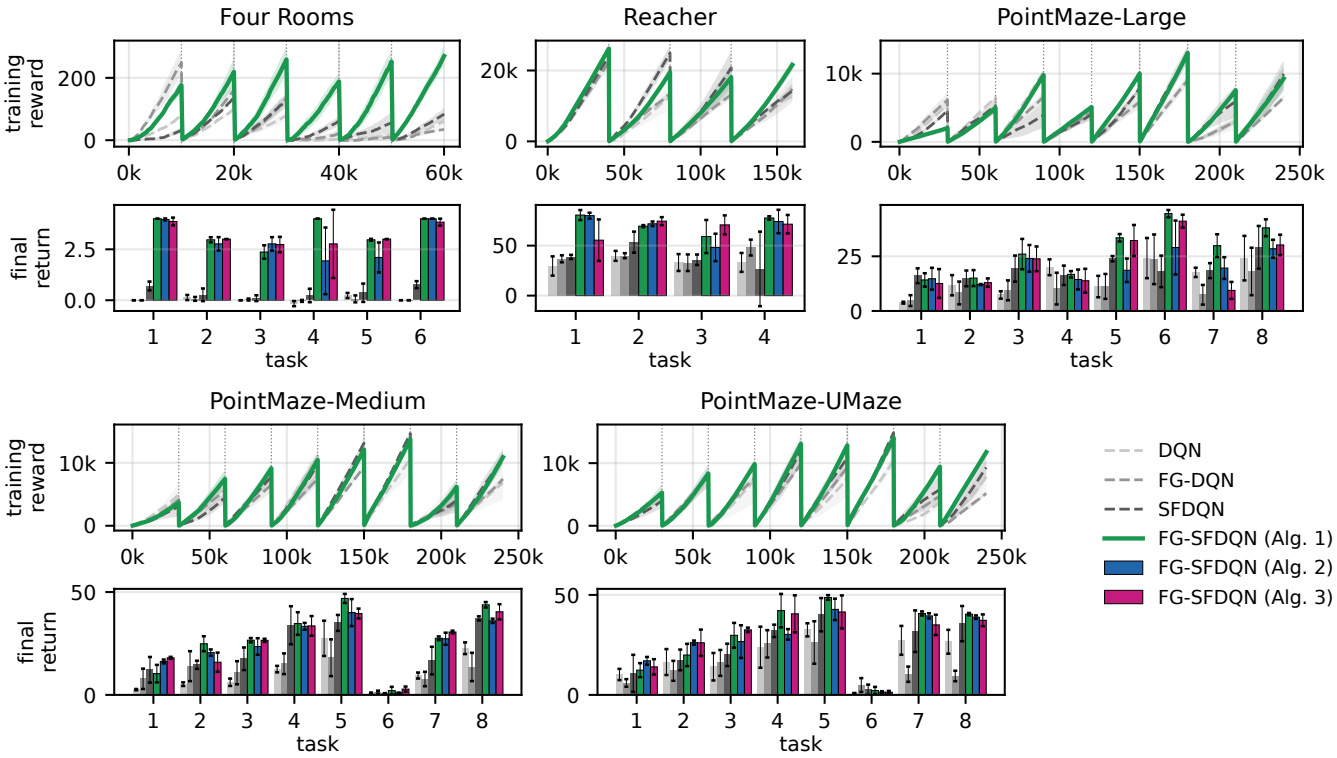



Fig. 1: Results in the five domains (mean $\pm$ std over 3 seeds). For each domain, upper panel: cumulative training reward on the active task for the sequential methods; dotted lines mark the task switches, where the curves restart at zero. Lower panel: final evaluation per task after training on all tasks (mean return over 10 episodes of at most 100 steps).

all benchmarks. Single-network DQN baselines suffer from catastrophic forgetting across task switches (e.g., FG-DQN drops to 0.02 in Four Rooms), whereas FG-SFDQN reduces the final squared SF Bellman residual by 5–46$\times$ compared to SFDQN, yielding accurate SFs for reliable GPI transfer (Appendix C.4).

*3) Effect of averaging:* Averaging reduces the double-sampling bias $V$, which is non-zero only when the next state of a pivot is random (Proposition 1). Consistent with Proposition 1, in the deterministic domains ($V = 0$) target averaging with $N \in \{5, 10, 20\}$ matches single-sample FG-SFDQN while reducing standard deviation by up to $8\times$ (Appendix C.5). We therefore evaluate averaging on *slippery* variants of Four Rooms and Reacher, in which the action of the agent is replaced, with probability $p \in \{0.2, 0.3\}$, by a uniformly random action. We compare Algorithm 1 and Algorithm 2 with their averaged counterparts ($N = 5$; Algorithm 3 for Algorithm 2) and with the baselines, using 3 seeds, 6 tasks of 30,000 steps in Four Rooms, and otherwise the settings of Table I. Since continuous states do not repeat in the buffer, Reacher draws the $N$ next states from the simulator.

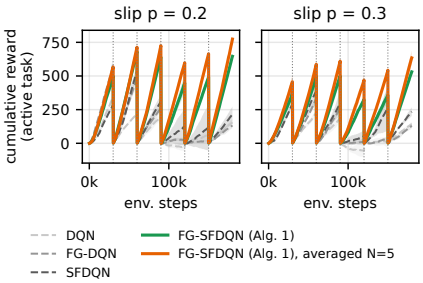
Under stochastic transitions ($V > 0$), target averaging consistently improves both sequential and randomized FG-SFDQN (Fig. 2; full tables in Appendices C.2–C.3). In slippery Four Rooms (Fig. 2a–b), averaging ($N = 5$) increases cumulative training reward by 19–28% for Algorithm 1 and 12–15% for Algorithm 2, while boosting final evaluation returns by 19–30% (reaching 3.79 at $p = 0.2$ and 3.19 at $p = 0.3$). All four FG-SFDQN variants achieve final returns above 2.67, whereas all baselines collapse below 0.55. In slippery Reacher (Fig. 2c–d), averaging improves final evaluation return by 17–21% for Algorithm 1 (reaching 69.3 and 61.7) and by 27–31% for Algorithm 2 (reaching 68.1 and 63.0). While SFDQN collects comparable training reward in slippery Reacher, its semi-gradient estimates degrade across tasks, resulting in poor final retention (42.3 and 46.6); averaged FG-SFDQN outperforms SFDQN in final evaluation by 32–64%.

*4) Computational overhead:* Full-gradient differentiation adds no per-step computational overhead over semi-gradient SFDQN: on PointMaze-UMaze, Algorithm 1 matches the mean step time of SFDQN (4.34 vs. 4.47 ms) with a lower median (2.67 vs. 4.19 ms), while $N = 5$ target averaging adds only 18% overhead (5.11 ms mean, 3.85 ms median; Appendix C.6). Overall, FG-SFRQL substantially outperforms semi-gradient baselines in both Bellman residual reduction and multi-task transfer returns while incurring negligible computational overhead.

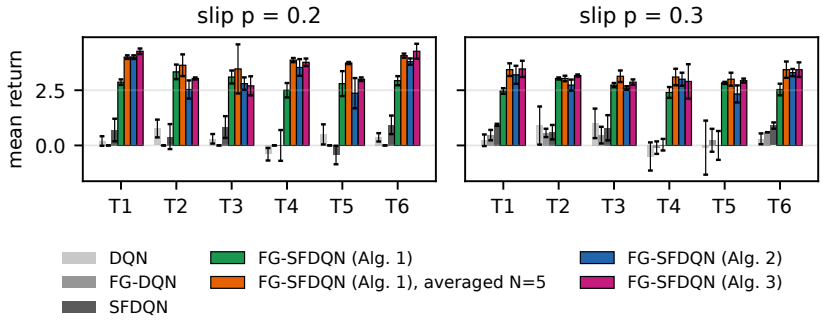
## References


[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, Cambridge, 1998, vol. 1, no. 1.

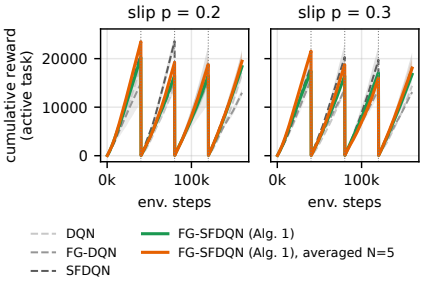
[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.



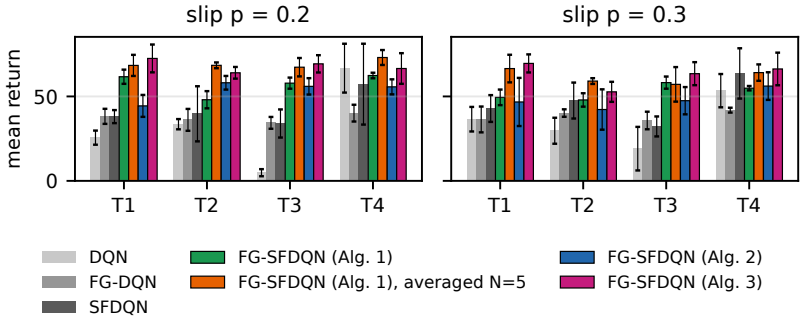

(a) Four Rooms: training reward per task




(b) Four Rooms: final evaluation per task




(c) Reacher: training reward per task




(d) Reacher: final evaluation per task

Fig. 2: Slippery Four Rooms (top) and Reacher (bottom), mean $\pm$ std over 3 seeds. Panels (a) and (c) show the cumulative training reward of the sequential methods; dotted lines mark the task switches, where the curves restart at zero. Totals are given in Appendices C.2 and C.3.


[3] P. Dayan, "Improving generalization for temporal difference learning: The successor representation," *Neural computation*, vol. 5, no. 4, pp. 613–624, 1993.

[4] A. Barreto, W. Dabney, R. Munos, J. J. Hunt, T. Schaul, H. Van Hasselt, and D. Silver, "Successor features for transfer in reinforcement learning," *Advances in neural information processing systems*, vol. 30, 2017.

[5] L. Lehnert and M. L. Littman, "Successor features combine elements of model-free and model-based reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 196, pp. 1–53, 2020.

[6] H. Liu and P. Abbeel, "Aps: Active pretraining with successor features," in *International Conference on Machine Learning*. PMLR, 2021, pp. 6736–6747.

[7] V. Myers, C. Zheng, A. Dragan, S. Levine, and B. Eysenbach, "Learning temporal distances: Contrastive successor features can provide a metric structure for decision-making," in *International Conference on Machine Learning*. PMLR, 2024, pp. 37 076–37 096.

[8] M. Gimelfarb, A. Barreto, S. Sanner, and C.-G. Lee, "Risk-aware transfer in reinforcement learning using successor features," *Advances in Neural Information Processing Systems*, vol. 34, pp. 17 298–17 310, 2021.

[9] A. K. Jain, H. Wiltzer, J. Farebrother, I. Rish, G. Berseth, and S. Choudhury, "Non-adversarial inverse reinforcement learning via successor feature matching," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 38 826–38 853.

[10] L. Baird, "Residual algorithms: Reinforcement learning with function approximation," in *Machine learning proceedings 1995*. Elsevier, 1995, pp. 30–37.

[11] J. Tsitsiklis and B. Van Roy, "Analysis of temporal-diffference learning with function approximation," *Advances in neural information processing systems*, vol. 9, 1996.

[12] M. Loth and P. Preux, "A unified view of td algorithms; introducing full-gradient td and equi-gradient descent td," *arXiv preprint cs/0611145*, 2006.

[13] K. E. Avrachenkov, V. S. Borkar, H. P. Dolhare, and K. Patil, "Full gradient dqn reinforcement learning: A provably convergent scheme," in *Modern Trends in Controlled Stochastic Processes: Theory and Applications, V. III*. Springer, 2021, pp. 192–220.

[14] C. Reinke and X. Alameda-Pineda, "Successor feature representations," *Transactions on Machine Learning Research*, 2023. [Online]. Available: https://openreview.net/forum?id=MTFf1rDDEI

[15] D. P. Bertsekas and J. N. Tsitsiklis, "Neuro-dynamic programming: an overview," in *Proceedings of 1995 34th IEEE conference on decision and control*, vol. 1. IEEE, 1995, pp. 560–564.

[16] S. Zhang, H. D. Fernando, M. Liu, K. Murugesan, S. Lu, P.-Y. Chen, T. Chen, and M. Wang, "Sf-dqn: Provable knowledge transfer using successor feature for deep reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2024, pp. 58 897–58 934.

[17] J. Kim, S. Park, and G. Kim, "Constrained gpi for zero-shot transfer in reinforcement learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 4585–4597, 2022.

[18] M. Benaïm, J. Hofbauer, and S. Sorin, "Stochastic approximations and differential inclusions," *SIAM Journal on Control and Optimization*, vol. 44, no. 1, pp. 328–348, 2005.

[19] V. S. Borkar, *Stochastic approximation: a dynamical systems viewpoint*. Springer, 2008, vol. 2.

[20] M. Chevalier-Boisvert, B. Dai, M. Towers, R. Perez-Vicente, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," *Advances in neural information processing systems*, vol. 36, pp. 73 383–73 394, 2023.

[21] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016.

[22] R. de Lazcano, K. Andreas, J. J. Tai, S. R. Lee, and J. Terry, "Gymnasium robotics," 2024. [Online]. Available: http://github.com/Farama-Foundation/Gymnasium-Robotics